

AWS Synchronous vs. Asynchronous Order Processing Analysis

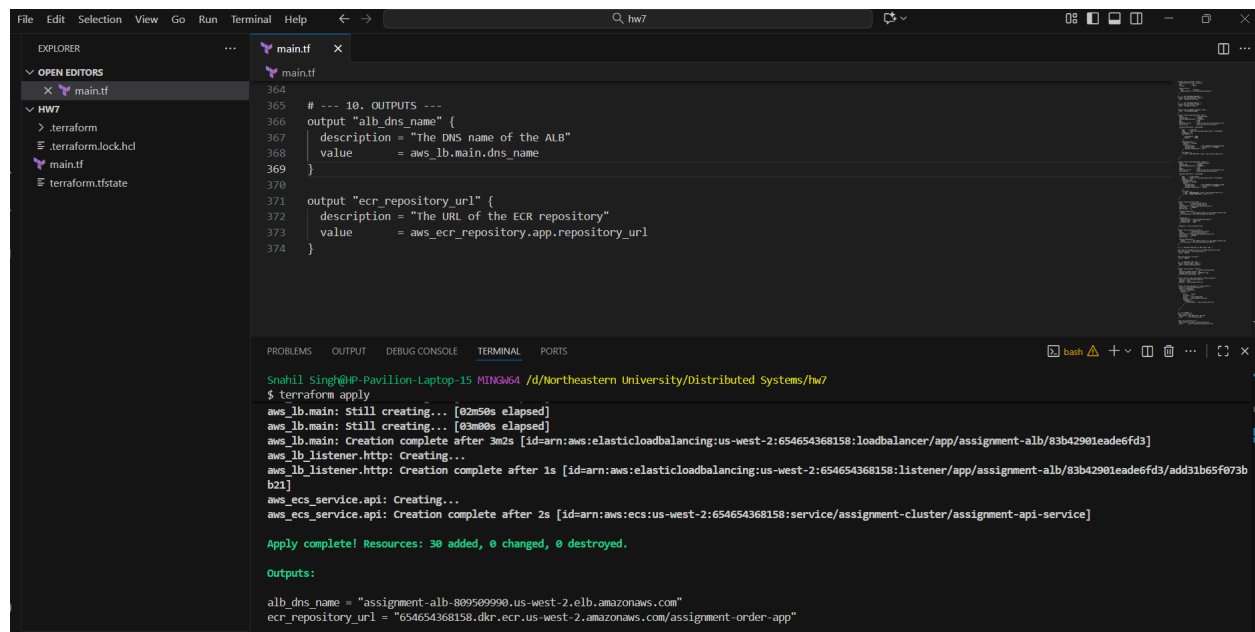
The goal was to understand the limitations of synchronous systems, implement an event-driven alternative using SNS and SQS with ECS workers, analyze scaling challenges, and evaluate a serverless Lambda alternative. Infrastructure was deployed using Terraform, applications were built in Go, and load testing was performed with Locust.

Synchronous vs. Asynchronous ECS Architecture

Infrastructure Setup

1. Terraform Deployment:

The infrastructure was successfully provisioned via terraform apply.



The screenshot shows a Visual Studio Code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project structure with files like `main.tf`, `terraform.lock.hcl`, and `terraform.tfstate`. The terminal window displays the output of a `terraform apply` command. The output shows the creation of various AWS resources, including an ALB, an ECR repository, and an ECS service. The final output shows the `alb_dns_name` and `ecr_repository_url` values.

```
main.tf
364
365 # --- 10. OUTPUTS ---
366 output "alb_dns_name" {
367   description = "The DNS name of the ALB"
368   value       = aws_lb.main.dns_name
369 }
370
371 output "ecr_repository_url" {
372   description = "The URL of the ECR repository"
373   value       = aws_ecr_repository.app.repository_url
374 }

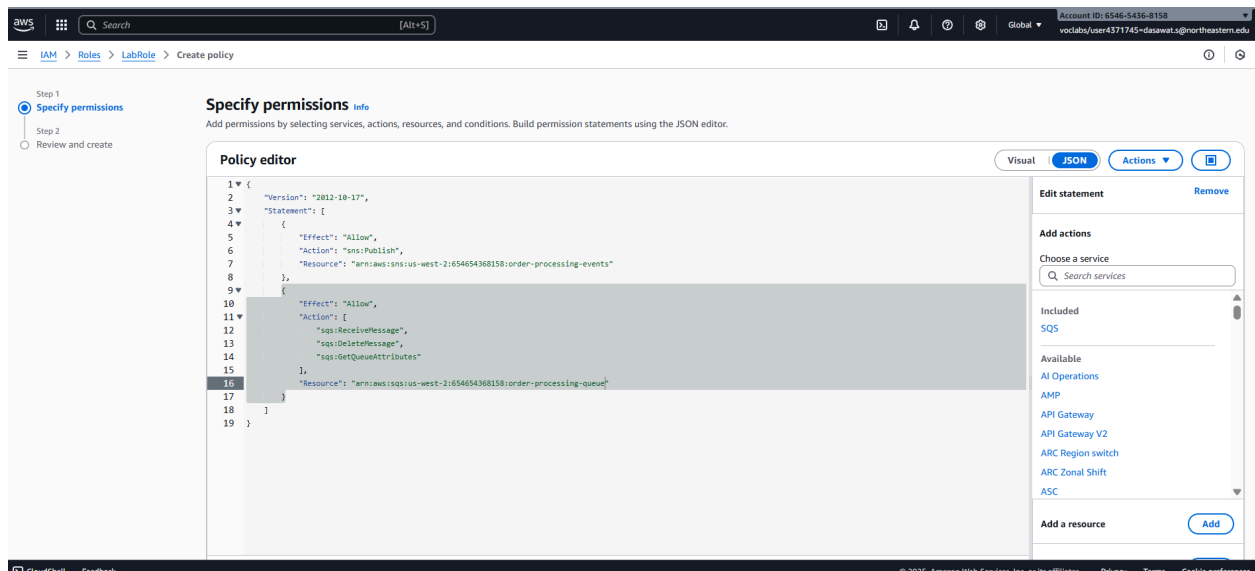
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
$ terraform apply
aws_lb.main: Still creating... [02m50s elapsed]
aws_lb.main: Still creating... [03m00s elapsed]
aws_lb.main: Creation complete after 3m2s [id=arn:aws:elasticloadbalancing:us-west-2:654654368158:loadbalancer/app/assignment-alb/83b42901eade6fd3]
aws_lb_listener.http: Creating...
aws_lb_listener.http: Creation complete after 1s [id=arn:aws:elasticloadbalancing:us-west-2:654654368158:listener/app/assignment-alb/83b42901eade6fd3/add31b65f073b2b]
aws_ecs_service.api: Creating...
aws_ecs_service.api: Creation complete after 2s [id=arn:aws:ecs:us-west-2:654654368158:service/assignment-cluster/assignment-api-service]

Apply complete! Resources: 30 added, 0 changed, 0 destroyed.

Outputs:
alb_dns_name = "assignment-alb-809509990.us-west-2.elb.amazonaws.com"
ecr_repository_url = "654654368158.dkr.ecr.us-west-2.amazonaws.com/assignment-order-app"
```

2. IAM Permissions:

Due to AWS Learner Lab limitations, IAM roles could not be created directly. The existing LabRole was used for both ECS Task Execution and Task Role. An inline policy (AssignmentSnsSqsPolicy-uswest2) was manually added to grant necessary permissions to the newly created SNS topic and SQS queue.



3. Application Deployment:

Two Go applications (order-api and order-worker) were containerized using Docker and pushed to the ECR repository. ECS Services were updated to deploy these containers.

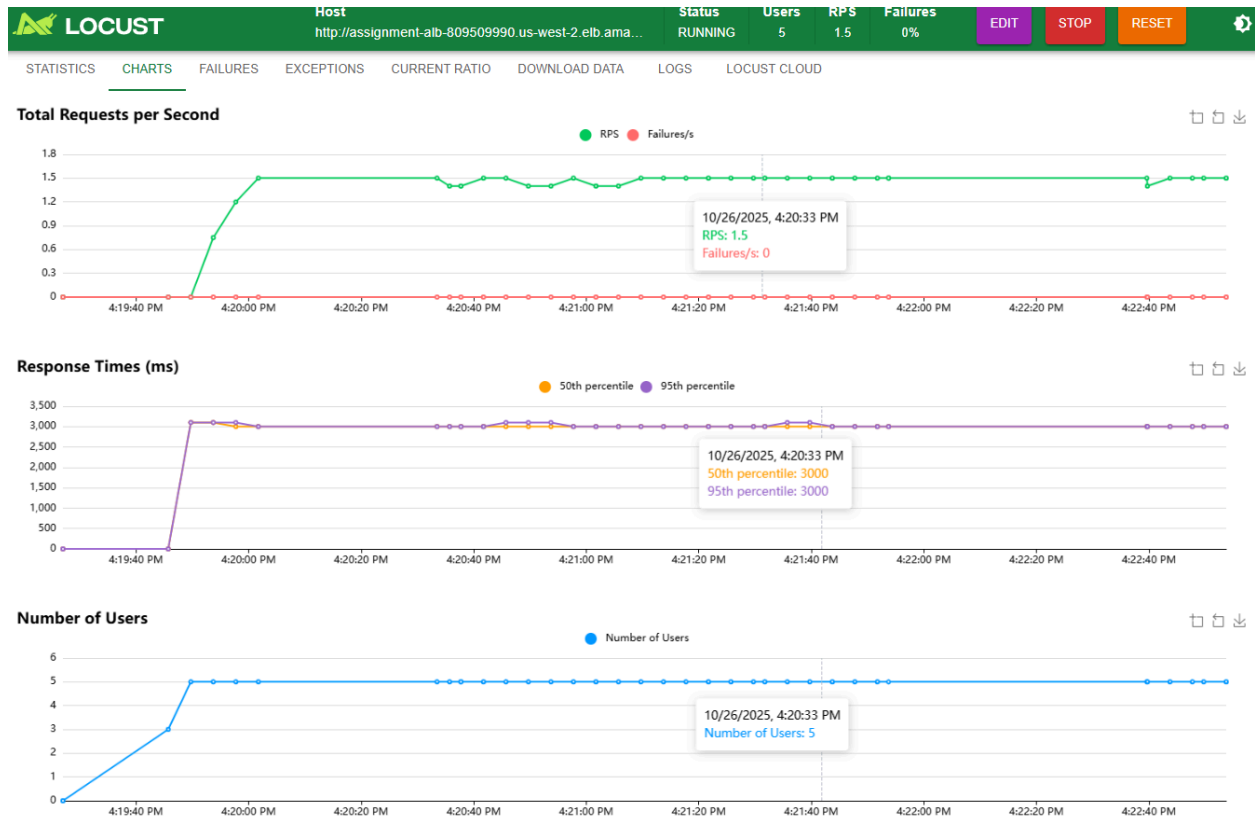
Phase 1: Build Your Current System (Synchronous)

The initial system implemented a synchronous API endpoint (/orders/sync) simulating a 3-second payment verification delay using a Go channel to enforce a single-threaded bottleneck.

1. Test Normal Operations (5 Users):

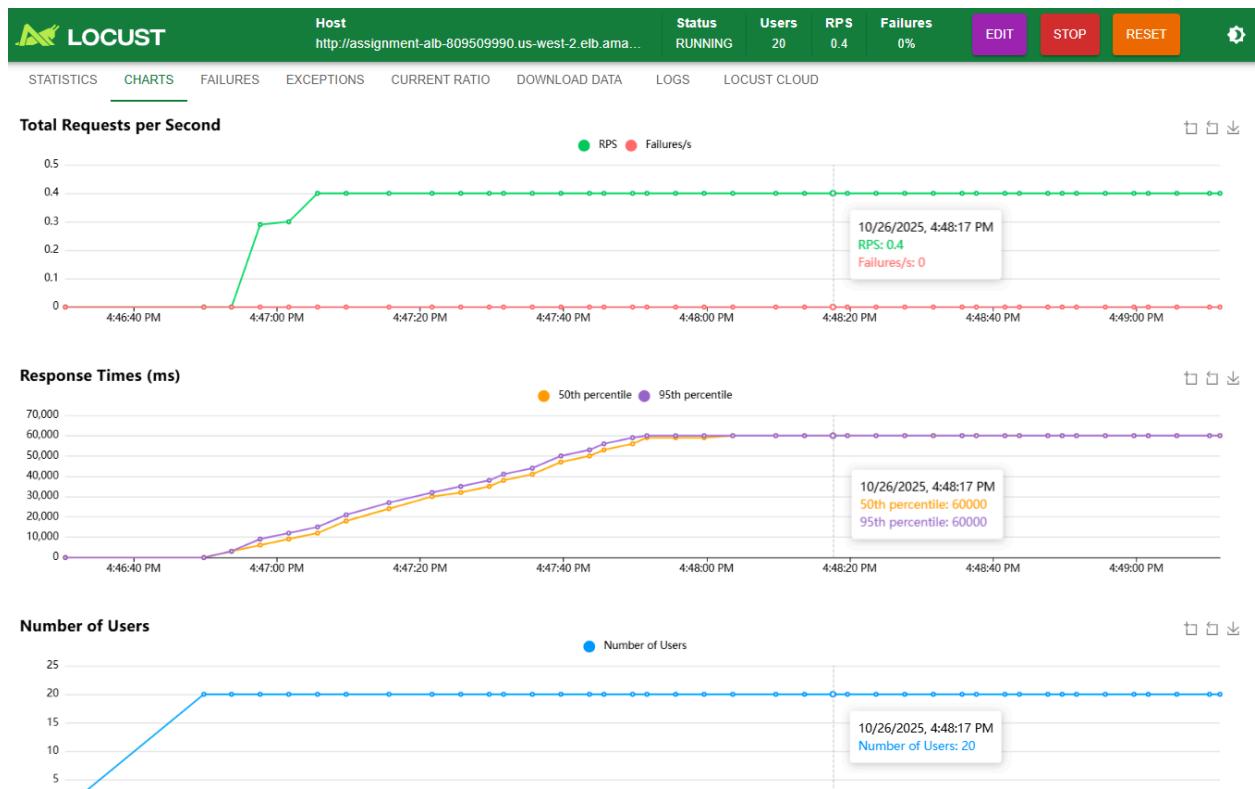
A Locust test simulated 5 concurrent users with a 1 user/second spawn rate for 30 seconds.

- **Analysis:** The system performed stably under normal load. Results showed 100% success, an average throughput of ~1.5 RPS, and a consistent 95th percentile response time of ~3000ms, matching the simulated delay.



2. Test Flash Sale (20 Users):

A Locust test simulated 20 concurrent users with a 10 user/second spawn rate for 60 seconds against the `/orders/sync` endpoint.



- **Analysis - What happens to your customers?** The system failed catastrophically. While Locust recorded 0% failures due to its long default timeout, the median and 95th percentile response times climbed linearly to **60,000ms (1 minute)**. Customers experienced extreme delays, rendering the site unusable. The maximum throughput achieved was only **0.4 RPS**.

Phase 2: Analyze the Bottleneck

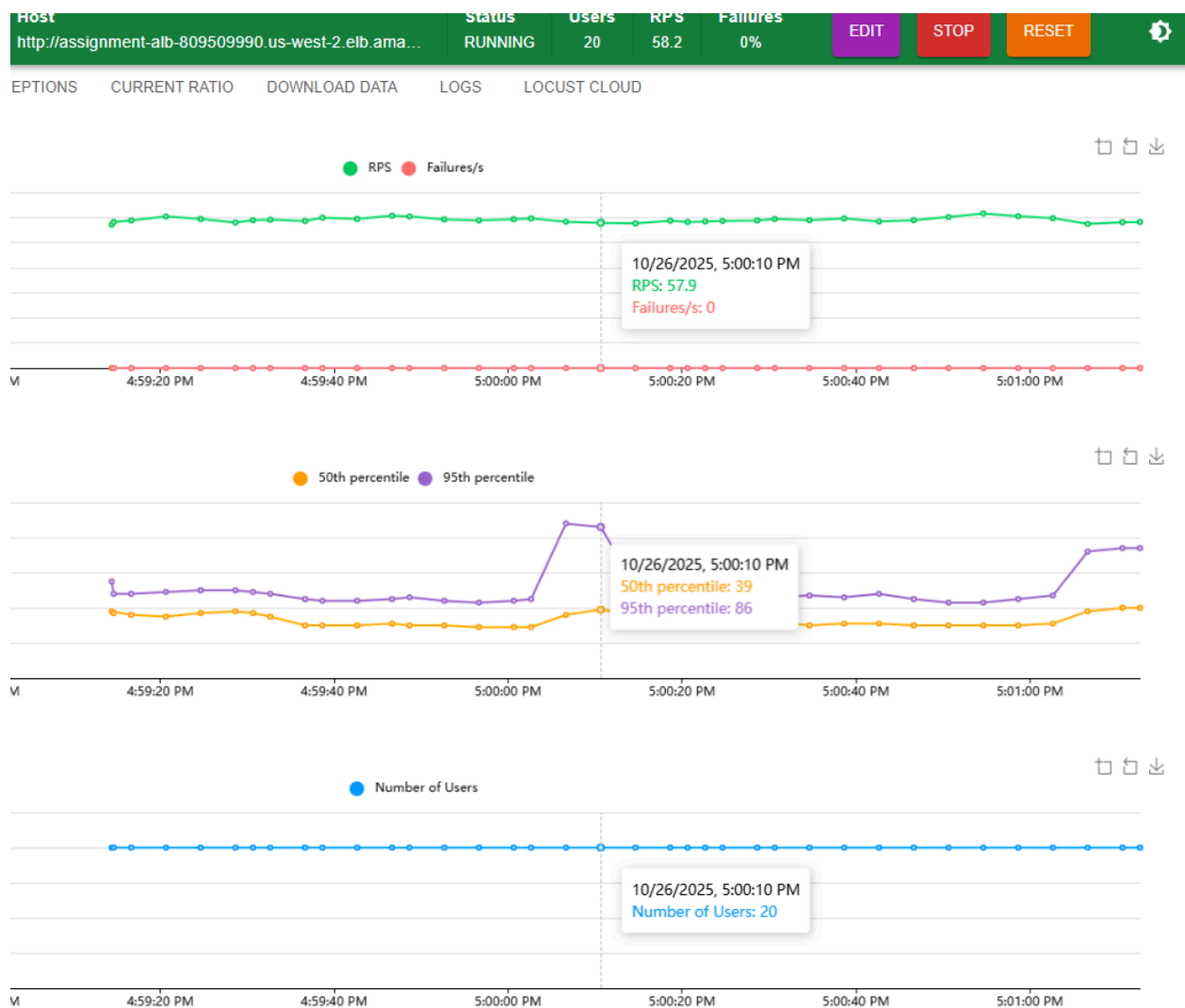
- **Payment Processor Speed:** 1 order per 3 seconds = **0.33 orders/second**.
- **Maximum System Throughput:** Confirmed by testing at **~0.4 orders/second**.
- **Flash Sale Demand:** ~58 orders/second (observed from async test).
- **Orders Lost/Delayed per Second:** $58 - 0.33 = 57.67$ orders per second.
- **The Problem:** The synchronous architecture forces customers to wait for the slow payment process. The only solution is to decouple order acceptance from processing.

Phase 3: The Async Solution

An asynchronous endpoint (/orders/async) was implemented. It accepts the order, publishes it to an SNS topic (order-processing-events), and immediately returns a 202 Accepted response (<100ms). An SQS queue (order-processing-queue) subscribed to the topic, feeding an ECS worker service (order-worker) running 1 worker goroutine initially.

1. Test Flash Sale (20 Users - Async):

The same 20-user Locust test was run against the /orders/async endpoint for 60 seconds.

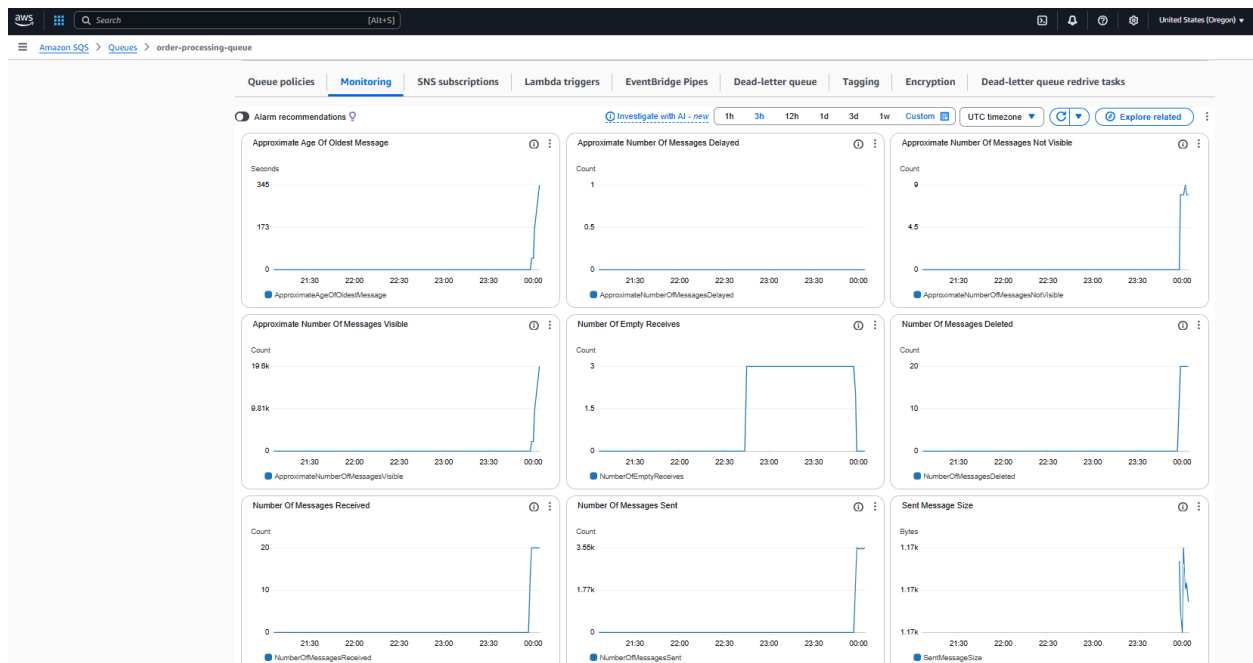


- **Analysis:** The results were dramatically different. The system achieved **100% success** with a throughput of **58.2 RPS** and a median response time of **39ms** (95th percentile: 86ms).
- **Answer to Question:** The asynchronous approach accepted **145 times more orders** (58.2 RPS / 0.4 RPS) compared to the synchronous approach during the flash sale simulation, providing an instant response to the customer.

Phase 4: The Queue Problem

While the API was fixed, the accepted orders overwhelmed the single background worker.

- **Order Acceptance Rate:** ~58 orders/second.
- **Single Worker Processing Rate:** 0.33 orders/second.
- **Queue Growth Rate:** 57.67 messages per second.
- **CloudWatch Monitoring:** SQS metrics confirmed the rapid queue buildup.



- **Analysis:** The test generated ~3,500 messages in 60 seconds. The queue

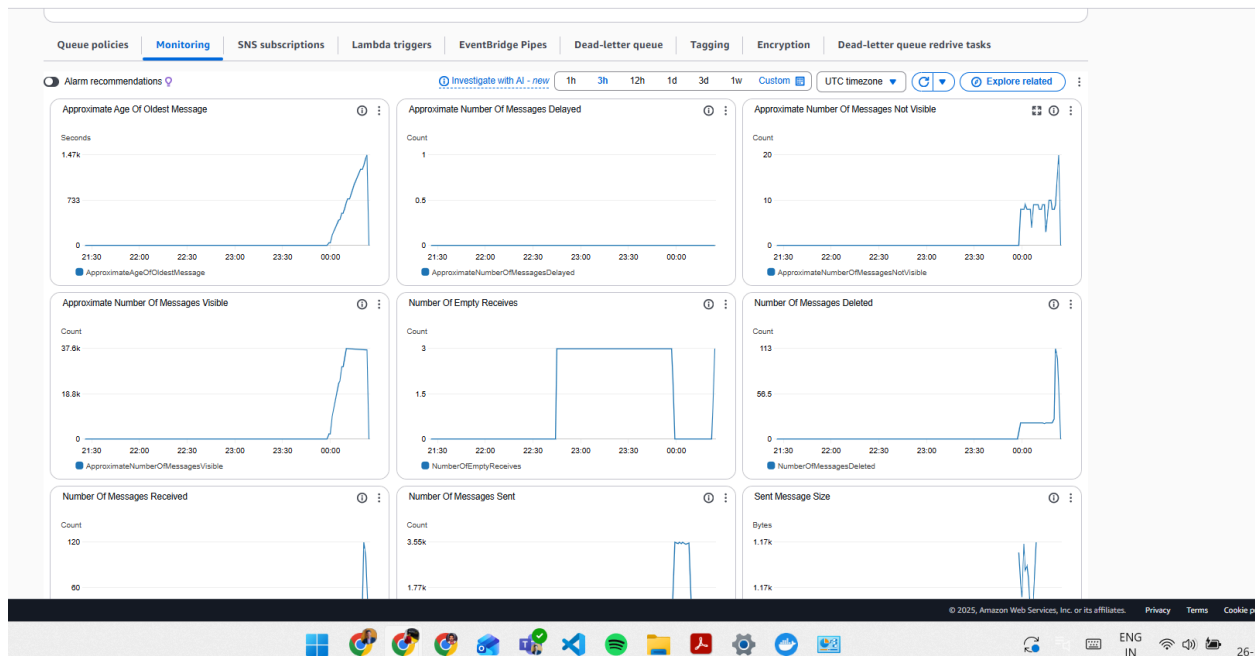
grew rapidly, leading to significant delays in actual order fulfillment (estimated ~3 hours to clear).

Phase 5: Scale Your Workers

The number of concurrent worker goroutines within the single ECS task was scaled to handle the load.

1. Test 1: 5 Goroutines:

The `WORKER_GOROUTINES` environment variable was set to 5, and the flash sale test was repeated after purging the queue.



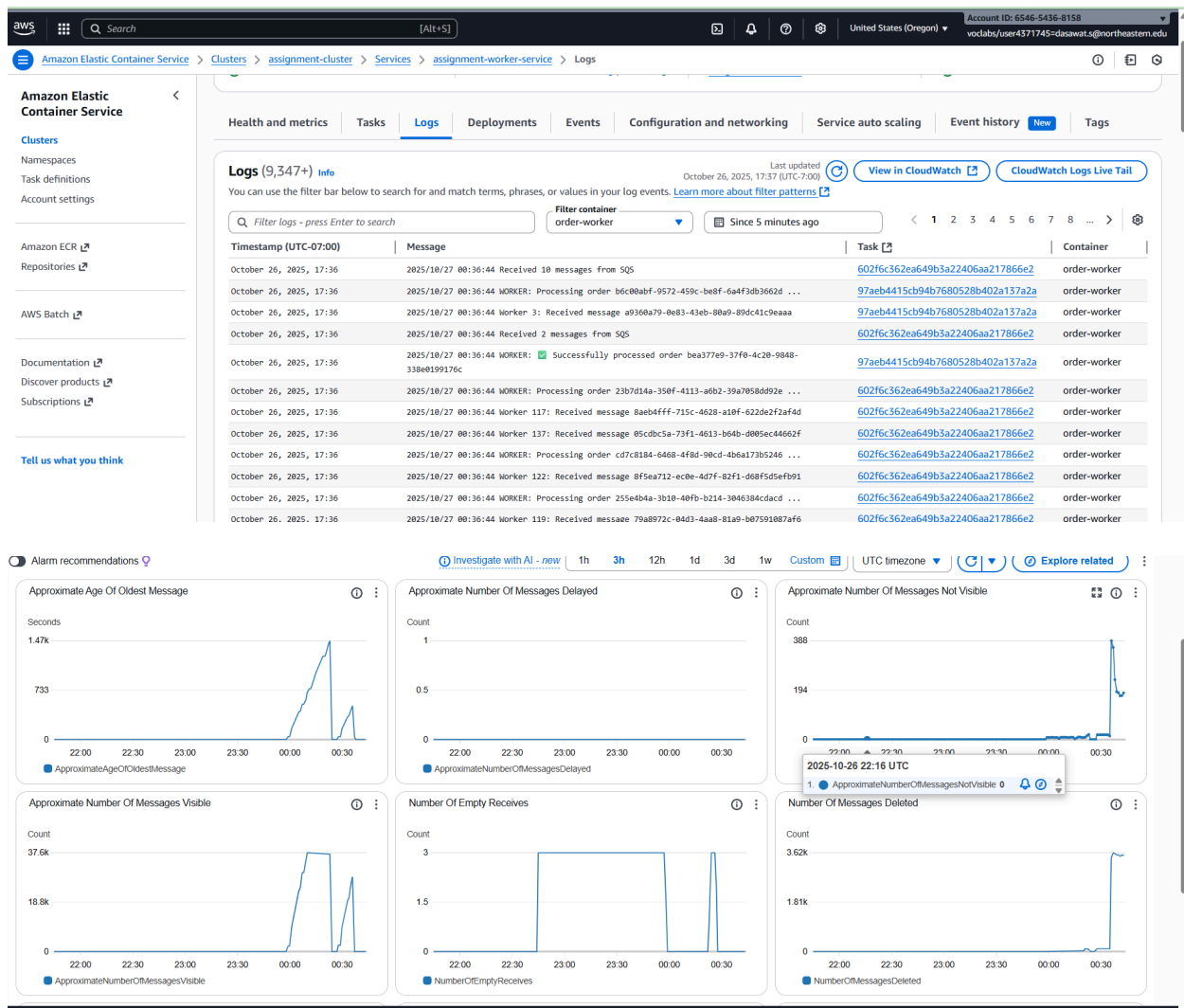
- **Analysis:** Processing rate increased to 1.65 orders/sec (\$5 \times 0.33\$). However, this was still far below the ~58 orders/sec ingestion rate. The queue backlog continued to grow rapidly, reaching over 37k messages across multiple test runs before being purged.

2. Calculation: Find the Balance:

To handle ~60 orders/sec with workers processing at 0.33 orders/sec, the minimum number of workers required is $60 / 0.33 = 182$ workers.

3. Test 2: 182 Goroutines:

The `WORKER_GOROUTINES` environment variable was set to 182, the queue was purged, and the flash sale test was repeated.



- **Analysis:** Success! With 182 workers, the processing capacity (~60.06 orders/sec) matched the ingestion rate (~58 orders/sec). The SQS chart shows that while messages briefly queued during the 60-second test, the backlog was cleared almost immediately afterward, preventing significant customer delays. The Number Of Messages Deleted chart spiked to ~3.6k, confirming all test messages were processed quickly.

Part III: Lambda (Serverless) Alternative

The SQS queue and ECS worker service were replaced with a single AWS Lambda function (assignment-order-processor) subscribed directly to the SNS topic.

Setup

A Go Lambda function was created using the provided.al2 (Amazon Linux 2) runtime, configured with 512MB memory and a 10-second timeout. It was assigned the LabRole for permissions and triggered by the existing order-processing-events SNS topic. The ECS worker service was scaled down to 0 tasks.

Observation

Several test orders were sent via the /orders/async endpoint. CloudWatch logs were examined.

The screenshot shows the AWS CloudWatch console. The breadcrumb navigation is: CloudWatch > Log groups > /aws/lambda/assignment-order-processo > 2025/10/27/[SLATEST]c3291ec5e081450490850157888af8f0. The left sidebar shows the 'Log groups' section expanded. The main area is titled 'Log events' and contains a search bar and a table of log events. The table has columns for 'Timestamp' and 'Message'. The messages show the function's execution details, including the start of the process, receipt of an SNS message, successful processing of an order, and the end of the process. The function is identified as 'assignment-order-processo' and is using the 'provided.al2' runtime.

Timestamp	Message
2025-10-27T01:05:21.348Z	INIT_START Runtime Version: provided.al2.v128 Runtime Version ARN: arn:aws:lambda:us-west-2::runtime:cc894672bc585ed6a7f4a86074d252eaa683a71b131b2cd8bf51a2..
2025-10-27T01:05:21.427Z	START RequestId: 806ec3bb-4975-40fb-8316-cee4349238da Version: \$LATEST
2025-10-27T01:05:21.428Z	2025/10/27 01:05:21 Received SNS message ID = 052d1762-d8a4-5851-b7d4-761af35a63b
2025-10-27T01:05:21.428Z	2025/10/27 01:05:21 LAMBDA: Processing order 14062556-e288-4f7b-b935-33d080693afd ...
2025-10-27T01:05:24.428Z	2025/10/27 01:05:24 LAMBDA: [X] Successfully processed order 14062556-e288-4f7b-b935-33d080693afd
2025-10-27T01:05:24.430Z	END RequestId: 806ec3bb-4975-40fb-8316-cee4349238da
2025-10-27T01:05:24.430Z	REPORT RequestId: 806ec3bb-4975-40fb-8316-cee4349238da Duration: 3002.59 ms Billed Duration: 3088 ms Memory Size: 128 MB Max Memory Used: 19 MB Init Duration: ...
2025-10-27T01:05:24.570Z	START RequestId: 5c351d86-7be5-421e-9562-d49833d4c3c9 Version: \$LATEST
2025-10-27T01:05:24.571Z	2025/10/27 01:05:24 Received SNS message ID = 2118969e-016f-5211-8618-f7e6c6fbac7b
2025-10-27T01:05:24.571Z	2025/10/27 01:05:24 LAMBDA: Processing order c6865268-ab7a-4c11-875b-6ceac9df0198 ...
2025-10-27T01:05:27.573Z	2025/10/27 01:05:27 LAMBDA: [X] Successfully processed order c6865268-ab7a-4c11-875b-6ceac9df0198
2025-10-27T01:05:27.574Z	END RequestId: 5c351d86-7be5-421e-9562-d49833d4c3c9
2025-10-27T01:05:27.574Z	REPORT RequestId: 5c351d86-7be5-421e-9562-d49833d4c3c9 Duration: 3003.55 ms Billed Duration: 3004 ms Memory Size: 128 MB Max Memory Used: 26 MB

The screenshot shows the AWS Lambda console for the 'assignment-order-processo' function. The breadcrumb navigation is: Lambda > Functions > assignment-order-processo. The function is in the 'Configuration' tab. The 'Function overview' section shows the function's name, layers (0), and triggers (SNS). The 'Description' section shows the function's ARN and URL. The 'Configuration' section shows the function's runtime (provided.al2), memory (512 MB), and timeout (10 seconds).

Function overview

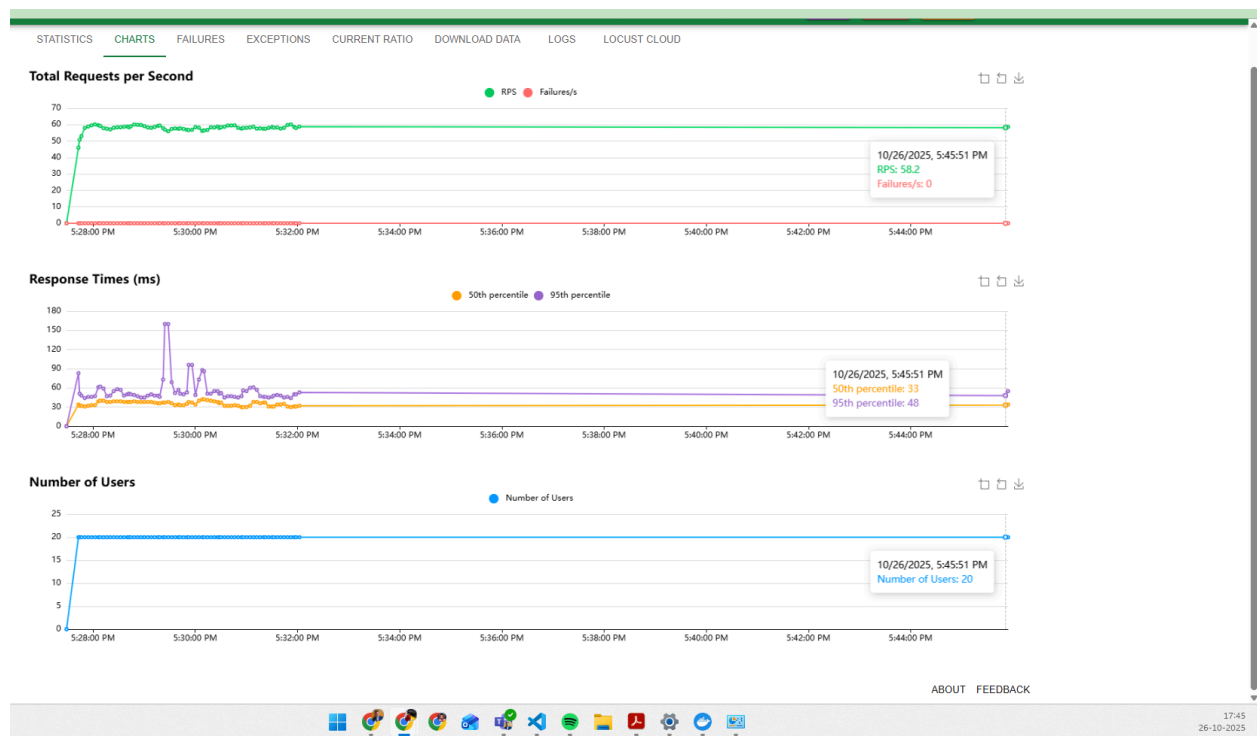
- Diagram
- Template
- assignment-order-processo
- Layers (0)
- SNS
- + Add trigger

Description

-
- Last modified: 2 minutes ago
- Function ARN: arn:aws:lambda:us-west-2:654654368158:function:assignment-order-processo
- Function URL: Info

Configuration

- Code
- Test
- Monitor
- Configuration
- Aliases
- Versions



Analysis

- **Cold Starts:** Occurred on the first invocation after deployment/idle period. The observed Init Duration was **407.47 ms**. This adds ~13.6% overhead to the first 3-second processing task but does not affect subsequent warm invocations (which took ~3005 ms).
- **Cost Comparison:**
 - ECS (2 tasks): ~\$17/month (fixed).
 - Lambda: \$0/month (within the free tier of 1M requests + 400k GB-seconds, covering ~267k orders/month for this workload).
- **Trade-offs:**
 - **Gained:** Zero operational overhead (no queue/worker management, auto-scaling), pay-per-use pricing (effectively free for low volume).
 - **Lost:** SQS buffering/retry guarantees (SNS retries twice then discards), SQS batching capabilities, and potential minor cold start latency.

Recommendation

Switch Recommendation: Yes, the startup should switch to Lambda.

Justification: For a startup, the Lambda architecture offers significant advantages. The **\$0 cost** within the generous free tier and the **elimination of operational overhead** (no need to manage ECS scaling, monitor queues, patch servers) are compelling. The development team can focus on features, not infrastructure management. The observed 407ms cold start is acceptable for this background task. While the lack of SQS guarantees is a risk, it can be mitigated with a Dead Letter Queue (DLQ) for failed SNS messages and robust logging within the Lambda function. The benefits of cost savings and simplified operations heavily outweigh the manageable risks for this specific workload and startup context.

Conclusion

This project demonstrated the critical limitations of synchronous architectures under load and the significant benefits of adopting an event-driven, asynchronous approach using AWS SNS and SQS. While the asynchronous ECS worker pattern solved the initial API bottleneck, it introduced new challenges related to queue management and worker scaling, requiring careful calculation (182 workers) to balance ingestion and processing rates.

Finally, evaluating the serverless Lambda alternative highlighted a compelling trade-off: **sacrificing the robustness of SQS queuing for dramatic cost savings and the complete elimination of operational overhead. For the target startup scenario, the Lambda architecture proved to be the superior choice due to its cost-effectiveness and operational simplicity.**